«МістоРух»

Симулятор і аналізатор міської транспортної мережі

Демонстрація 1 — Спринт 1 (тижні 1–4)

Жидик Артем · Мельникович Ігор · Чайківський Захар · Лагута Олександра · Дрижирук Альона

github.com/Artzh13/Citymove

Розповідає: Жидик Артем

Порядок доповіді

Розповідає: Жидик Артем

Розповідаємо по тижнях; у кожному тижні — від ролі A до E

Тиждень	A · domain.py	B · dataio.py	C · core.py	D · analytics.py	E · cli.py, infra.py
1	Жидик · сл. 5	Мельникович · сл. 6	Чайківський · сл. 7	Лагута · сл. 8	Дрижирук · сл. 9
2	Дрижирук · сл. 10	Жидик · сл. 11	Мельникович · сл. 12	Чайківський · сл. 13	Лагута · сл. 14
3	Лагута · сл. 15	Дрижирук · сл. 16	Жидик · сл. 17	Мельникович · сл. 18	Чайківський · сл. 19
4	Чайківський · сл. 20	Лагута · сл. 21	Дрижирук · сл. 22	Жидик · сл. 23	Мельникович · сл. 24–25

- Ролі ротувались щотижня: кожен за 4 тижні побував у 4 різних ролях і тримав 4 різні модулі.
- Кожен розповідає про **свій тиждень у своїй ролі** — що зробив, що не встиг, які проблеми знайшов і що передав наступному власнику. Усе це — з журналів передачі docs/HANDOVER-A…E.txt.
- Наприкінці: тести — Чайківський (сл. 26), як ми працювали — Дрижирук (сл. 27), обмеження і план на Спринт 2 — Лагута (сл. 28).

Мета і дані

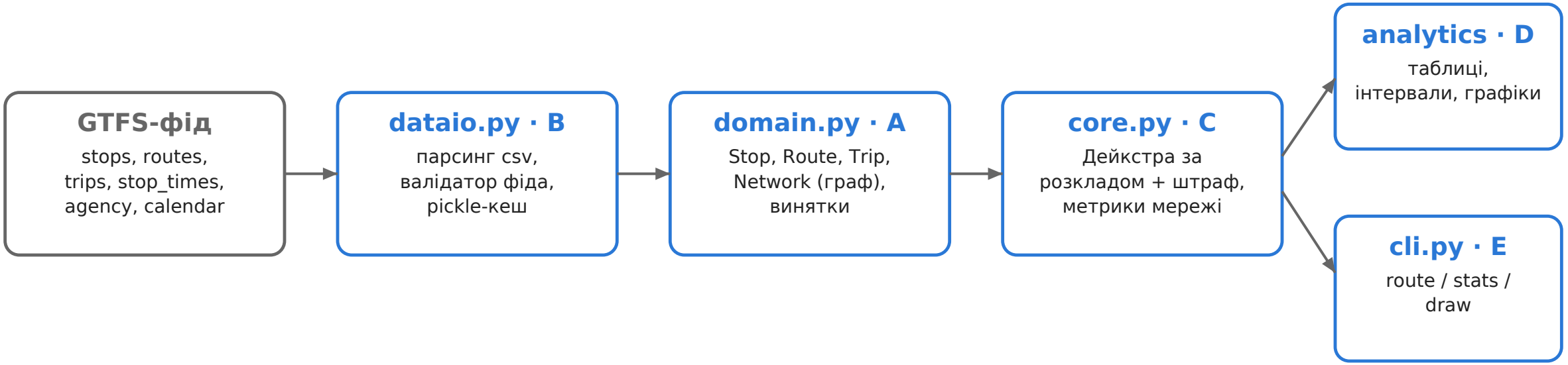
Розповідає: Жидик Артем

- **Мета Спринту 1:** консольна команда завантажує GTFS-фід, будує мережу, знаходить найшвидший шлях між двома зупинками з урахуванням пересадок і друкує покроковий маршрут із часом.
- **Реальні дані:** GTFS-фід КП «Київпастранс» (версія 20261006)
  - 1 492 зупинки · 162 маршрути (102 автобуси, 43 тролейбуси, 17 трамваїв) · 24 393 рейси · 104 123 рядки stop_times
- **Тестові дані з відомою відповіддю:**
  - мініфід tests/fixtures/mini_gtfs — 4 зупинки, маршрут з однією пересадкою
  - синтетичні мережі в core.py — радіально-кільцева та бенчмарк пересадок
- **Стек:** Python 3.11+, стандартна бібліотека (csv, heapq, argparse, logging, zoneinfo, pickle) + matplotlib; requirements.txt із закріпленими версіями. NumPy/Pandas і GUI — Спринт 2.

Архітектура

Розповідає: Жидик Артем

Пакет src/citymove/ — кожен модуль відповідає одній ролі; контракт між модулями — docs/spec.md



Час у всій системі — **хвилини від опівночі** (рейси після опівночі мають > 1440). Ребро графа: (to_stop_id, departure_min, travel_min, trip_id).

infra.py + config.json · E

logging → citymove.log · @timed · @log_call · @memoize · load_config
штраф за пересадку, швидкість ходьби, шлях до фіда

Тиждень 1 · Роль A: об'єктна модель

Розповідає: Жидик Артем

domain.py, docs/spec.md · журнал передачі: docs/HANDOVER-A.txt

- Створено класи предметної області **Stop, Route, Trip, Passenger** і **Network** — на них далі побудована вся програма.
- **Stop**: stop_id лише для читання (@property); name, lat, lon задаються через сетери з перевіркою — порожня назва або координати поза [-90, 90] / [-180, 180] дають ValueError.
- **Route** — id, номер для пасажира (short_name), вид транспорту і список stop_id у порядку проходження. **Trip** — рейс маршруту: start_time (не може бути від'ємним), service_id і розклад [(stop_id, прибуття, відправлення)] у хвилинах. **Passenger** — id та ім'я (порожнє → «Unknown»).
- Ієрархія транспорту: абстрактний **TransportUnit** (ABC, абстрактний name()) → **Bus, Tram, Trolleybus, Metro**. Швидкість, місткість і вартість у кожного свої й зашиті в конструкторі: автобус 20 км/год, 80 місць, 15 грн; трамвай 16 / 150 / 8; тролейбус 18 / 100 / 12; метро 40 / 1000 / 20.
- Ієрархія винятків від **NetworkError**: StopNotFoundError, RouteNotFoundError, DisconnectedNetworkError, ScheduleError — інші модулі ловлять помилки мережі одним except.
- **Network** — контейнер: stops, routes, trips (словники за id), edges — список суміжності для Дейкстри, transfer_time = 5 хв.
- Контракт для команди описано в docs/spec.md: поля класів, формат ребер, винятки.
- 3 юніт-тести: властивості автобуса, валідація зупинки, створення рейсу.

З журналу передачі

Не зроблено: Network ще не повноцінна композиція: немає додавання і вилучення вузлів з перевіркою зв'язності.

Відомі проблеми: Network поки лише контейнер даних без логіки керування графом.

Наступний крок: add_node(stop) і remove_node(stop_id) з перевіркою зв'язності графа при видаленні.

Новому власнику: edges зберігають trip_id і час у хвилинах від опівночі — під потреби ролі C; час на пересадку за замовчуванням 5 хв.

Тиждень 1 · Роль B: парсинг GTFS

Розповідає: Мельникович Ігор

dataio.py · журнал передачі: docs/HANDOVER-B.txt

- Закрито пункт «читання GTFS стандартним модулем csv, зіставлення сутностей за ключами, побудова структур ролі A».
- **read_csv** — генератор над csv.DictReader: читає файл построчно, кодування utf-8-sig (у київських файлах на початку є BOM).
- **load_stops, load_routes, load_trips, load_stop_times** — кожен файл перетворюється на об'єкти з domain.py; зв'язки шукаються за ключами: route_id у trips.txt, trip_id і stop_id у stop_times.txt.
- stop_times групуються за trip_id і сортуються за stop_sequence — так кожен рейс отримує свій розклад.
- **make_transport**: route_type → клас транспорту (0 — трамвай, 1 — метро, 3 — автобус, 11 — тролейбус).
- **build_network** збирає Network: між сусідніми зупинками кожного рейсу — ребро (to_stop_id, відправлення, час у дорозі, trip_id); ребра відсортовані за часом відправлення. Усе разом викликає **load_network(folder)**.
- Наперед частково зроблено пункт тижня 2: **time_to_minutes**('25:10:00') → 1510; **fix_midnight** — якщо після «23:58» іде «00:05», до решти розкладу додається 1440 хв (київський фід так пише рейси після опівночі).
- 4 юніт-тести: переведення часу, некоректний час, fix_midnight, завантаження мережі.

З журналу передачі

Не зроблено: Часові зони; валідація фіда (обов'язкові поля, посилальна цілісність, звіт про помилки).

Відомі проблеми: Зупинка зі stop_times, якої немає в stops.txt, не перевіряється. Одна зупинка з порожньою назвою або кривими координатами зупиняє все завантаження. Маршрути 9к і 19т не мають рейсів.

Наступний крок: Часові зони з agency.txt, далі — валідатор фіда.

Новому власнику: Час — у хвилинах від опівночі, після опівночі > 1440: ролі C не можна брати час за модулем 1440. Network поки заповнюється напряму — add_node ще немає.

Тиждень 1 · Роль C: алгоритм Дейкстри

Розповідає: Чайківський Захар

core.py · журнал передачі: docs/HANDOVER-C.txt

- Створено клас **TransportRouter**: конструктор приймає Network з domain.py і нумерує зупинки та рейси (stop_id → індекс, trip_id → індекс), щоб алгоритм працював з індексами, а не з рядками.
- **build_graph** будує граф: для кожної зупинки список ребер (час відправлення, час у дорозі, рейс, наступна зупинка) з усіх рейсів; ребра відсортовані за часом відправлення.
- **shortest_path_from**(A, B, time_0) знаходить найраніший момент, коли можна дістатися з A до B, якщо бути на A о time_0. Дейкстра на черзі з пріоритетом **heapq**.
- Стан алгоритму — пара **(зупинка, рейс)**: так видно, чи пасажир їде далі тим самим рейсом, чи пересідає; за пересадку враховується штраф.
- Рейс, який уже відійшов (відправлення раніше за поточний час), пропускається — сісти можна лише на транспорт, що ще не поїхав.
- Додатковий стан **M** — пасажир ще не сів у транспорт.

  Немає зупинки → StopNotFoundError, шляху немає → RouteNotFoundError.
- Цей граф і стан (зупинка, рейс) лишились основою пошуку маршруту в проєкті.
-

З журналу передачі

Не зроблено: Метрики мережі; перевірка коректності на синтетичній мережі з наперед відомими відповідями.

Було б добре додати: Стан day_of_week — маршрути залежать від дня тижня; для цього треба змінити Network у domain.py.

```
for dep, travel, trip_idx, nxt in graph[stop]:
    if dep < cur_time:
        continue        # уже поїхав
    if trip_idx == idx or idx == M:
        new = dep + travel
    else:               # пересадка
        new = dep + travel + penalty
    if new < dist[nxt][trip_idx]:
        dist[nxt][trip_idx] = new
        heappush(queue, (new, nxt, trip_idx))
```

Тиждень 1 · Роль D: консольні таблиці

Розповідає: Лагута Олександра

analytics.py · журнал передачі: docs/HANDOVER-D.txt

- Написано перші функції модуля аналітики — текстові таблиці для виводу в консоль.
- **stop_info_table**(stop, routes) — картка зупинки: назва, ID, координати (4 знаки після коми) і маршрути через кому; якщо маршрутів немає — «(Маршрутів немає)».
- **print_stop_info** — одразу друкує цю картку.
- **departure_board_table**(stop, trips) — табло відправлень: час, номер маршруту, вид транспорту і кінцева. Відбираються лише рейси маршрутів, що проходять через зупинку; якщо таких немає — «Рейсів на сьогодні більше немає»; невідома зупинка → StopNotFoundError.
- **print_depatrures_board** — друкує табло.
- Підхід «одна функція генерує рядок, друга друкує»: рядок легко перевірити в тесті. Так потім зроблено всі таблиці ролі D.
- 5 юніт-тестів: картка зупинки, порожній список маршрутів, табло, виняток для невідомої зупинки, властивості транспорту.

З журналу передачі

Не зроблено: Інтервали руху між рейсами; два графіки matplotlib; підсумковий консольний звіт.

Відомі проблеми: Щоб додати нове поле, треба міняти форматування табличок в обох функціях.

Наступний крок: Розрахунок інтервалів руху — у тому ж стилі: одна функція генерує, інша друкує.

Новому власнику: Для перевірки табличок створюй тимчасові класи й об'єкти, інакше не перевіриш.

```
========================================
        ІНФОРМАЦІЯ ПРО ЗУПИНКУ
========================================
| Назва:       Вокзал                  |
| ID:          A                       |
| Координати:  50.4400, 30.4900        |
| Маршрути:    1                       |
========================================
```

Тиждень 1 · Роль E: консольний інтерфейс

Розповідає: Дрижирук Альона

cli.py · журнал передачі: docs/HANDOVER-E.txt

- Створено модуль **cli.py** на **argparse**: create_parser з підкомандами (subparsers); команду вказувати обов'язково.
- Три базові команди: **route** --from --to --time — пошук найшвидшого маршруту; **stats** --feed — статистика GTFS-фіда; **draw** --out — збереження карти мережі у файл. Параметри обов'язкові, до кожного є підказка в --help.
- Обробники **handle_route, handle_stats, handle_draw** і **main()**, яка за назвою команди викликає потрібний обробник.
- Функції виводу **print_route_info** і **print_feed_stats** — їхні заголовки «ЗНАЙДЕНИЙ МАРШРУТ» і «СТАТИСТИКА МЕРЕЖІ» лишились у виводі CLI досі.
- Поки core.py і dataio.py в розробці, обробники працюють на заглушках — CLI уже можна запускати й тестувати.
- 3 тести: розбір аргументів route і stats, помилка, якщо не передано обов'язковий аргумент.

```
citymove route --from A --to B --time 08:00
citymove stats --feed data/gtfs_static
citymove draw  --out map.png
```

З журналу передачі

Не зроблено: Інтеграція з core.py, dataio.py — потрібні функції ще в розробці в інших учасників.

Відомі проблеми: Обробники виводять заготовлені тестові дані.

Наступний крок: Тиждень 2: глобальне логування (logging) і декоратори @timed та @log_call.

Тиждень 2 · Роль А: керування графом

Розповідає: Дрижирук Альона

domain.py · журнал передачі: docs/HANDOVER-A.txt

- Network став повноцінною композицією зупинок і ребер — з'явились методи керування графом.
- **add_node(stop)** — додає зупинку; якщо такий stop_id уже є — ValueError (захист від дублікатів); для нової зупинки одразу створюється порожній список ребер.
- **remove_node(stop_id)**: невідома зупинка → StopNotFoundError; інакше зупинка, її ребра і всі ребра інших зупинок, що ведуть до неї, тимчасово прибираються — з резервною копією.
- **_is_connected_graph** — обхід у ширину (BFS, черга) від першої зупинки: чи досяжні після видалення всі інші зупинки.
- Якщо граф розпався на частини — відкат з резервної копії і **DisconnectedNetworkError**: мережа лишається такою, як була.
- 3 юніт-тести: додавання й видалення, невідома зупинка, захист від розриву мережі.

З журналу передачі

Не зроблено: Усе заплановане на тиждень 2 виконано.

Відомі проблеми: remove_node чистить лише списки суміжності edges; синхронне видалення рейсів і маршрутів — на наступних етапах.

Наступний крок: Винятки для валідації і повна взаємодія моделі з парсером даних.

Новому власнику: remove_node сам перевіряє цілісність мережі через BFS і не дасть видалити зупинку, якщо граф розпадеться.

```
def _is_connected_graph(self, ids):
    visited = {ids[0]}
    queue = [ids[0]]
    while queue:
        cur = queue.pop(0)
        for edge in self.edges.get(cur, []):
            nb = edge[0]
            if nb in self.stops and \
                    nb not in visited:
                visited.add(nb)
                queue.append(nb)
    return all(n in visited for n in ids)
```

Тиждень 2 · Роль B: часові зони і валідатор

Розповідає: Жидик Артем

dataio.py · журнал передачі: docs/HANDOVER-B.txt

- **load_agency_timezone** читає agency.txt і перевіряє agency_timezone через zoneinfo; порожня або невідома зона → ScheduleError.
- **get_timezone** повертає об'єкт ZoneInfo; зона записується в **network.timezone** (для Києва — Europe/Kyiv); якщо agency.txt немає — UTC.
- Функція **validate_feed(folder, strict)** і клас **FeedValidationReport** (списки errors і warnings, is_valid).
- Перевіряється наявність файлів і обов'язкових колонок (stops: stop_id, stop_name, stop_lat, stop_lon; routes; trips; stop_times) та зв'язки: рейси → маршрути, розклад → зупинки; заодно — часова зона в agency.txt.
- **strict=False** — нічого не падає, повертається звіт зі списком проблем; **strict=True** — на першій помилці StopNotFoundError або RouteNotFoundError.
- 5 тестів у test_dataio.py: валідна й некоректна зона, валідний фід, фід без колонки і з битими зв'язками, load_network із часовою зоною. Усі тести проєкту зелені.

З журналу передачі

Не зроблено: Основне для ролі B закрито; лишилась підготовка тестових даних до Демо 1.

Відомі проблеми: Якщо в agency.txt кілька записів, береться перший. Криві координати чи порожня назва зупинки — ValueError з класу Stop.

Наступний крок: Прогнати валідатор на великому київському фіді: чи не підвисає і чи все завантажується.

Новому власнику: strict за замовчуванням False: функція не падає, а віддає звіт.

Тиждень 2 · Роль C: штраф за пересадку і метрики

Розповідає: Мельникович Ігор

core.py · журнал передачі: docs/HANDOVER-C.txt

- Алгоритм з тижня 1 перенесено у функції build_graph і **find_journey**; TransportRouter лишився обгорткою.
- **Штраф за пересадку** додається до вартості маршруту, але не блокує рух: сісти на рейс можна за фактичним часом. Пріоритет у черзі — «час + штрафи», тому при великому штрафі обирається варіант з меншою кількістю пересадок.
- Результат — **Journey**: вартість, фактичне прибуття, кількість пересадок і ділянки поїздки (legs) — по них друкується покроковий маршрут.
- Словники замість матриці «зупинки × рейси» — на київському фіді (1 492 × 24 393) матриця не влазить у пам'ять.
- Метрики мережі: **degree_centrality** — кількість сусідів / (N − 1); **network_diameter** і **average_shortest_path_length** — BFS з кожної зупинки, у перегонах.
- Незв'язна мережа: діаметр = inf, середня довжина → DisconnectedNetworkError (або ignore_disconnected=True — рахувати лише досяжні пари). **calculate_network_metrics** — усі метрики разом.
- 22 юніт-тести в tests/test_core.py: граф, пропущений рейс, штраф і вибір маршруту, legs, недосяжність, метрики на лінійній і зірковій мережах, порожня мережа.

З журналу передачі

Відомі проблеми: Рейси поки не фільтруються за днями тижня (service_id).

```
# вартість ребра
is_transfer = trip != M and t != trip
new_cost = cost + (arrive - now) \
         + (penalty if is_transfer
                    else 0)
# сісти можна, якщо dep >= now —
# за фактичним часом, не за cost
```

Тиждень 2 · Роль D: інтервали руху і перші графіки

Розповідає: Чайківський Захар

analytics.py · журнал передачі: docs/HANDOVER-D.txt

- **calculate_route_intervals**(trips, route_id) — бере рейси маршруту, сортує за start_time (у фіді вони бувають не по порядку) і рахує різниці між сусідніми рейсами — інтервали в хвилинах.
- **route_intervals_table** / **print_route_intervals** — таблиця: кількість рейсів, мінімальний, максимальний і середній інтервал; якщо рейс лише один — «Недостатньо рейсів для розрахунку інтервалів».
- **setup_plot_style** — спільний стиль графіків matplotlib: сітка, розмір 10 × 6, розміри шрифтів.
- **plot_interval_histogram** — гістограма інтервалів по всіх маршрутах; **plot_top_stops** — стовпчики топ-10 зупинок за кількістю з'єднань із підписами значень.
- **generate_console_report** — підсумковий звіт для Демо 1: кількість зупинок, маршрутів, рейсів і види транспорту.
- Тести з **mock**: plt.show() підмінено, щоб вікна графіків не блокували автоматичний прогін тестів.

З журналу передачі

Не зроблено: Pandas-конвеєр: groupby, merge і pivot_table для зведень «маршрут × година» та «день тижня × година».

Було б добре: Графікам потрібен Network з уже побудованими ребрами (edges) — для цього треба змінити дані, що надходять.

```
========================================
    ІНТЕРВАЛИ РУХУ: МАРШРУТ 1
========================================
| Кількість рейсів:        159         |
| Мінімальний інтервал:    3        хв |
| Максимальний інтервал:   369      хв |
| Середній інтервал:       9.0      хв |
========================================
трамвай №1, будні, один напрямок
```

Тиждень 2 · Роль E: логування і декоратори

Розповідає: Лагута Олександра

infra.py, cli.py · журнал передачі: docs/HANDOVER-E.txt

- Створено модуль **infra.py** з глобальним логуванням: **setup_logging** — рівень INFO, формат «час [рівень] логер: повідомлення», запис одночасно у файл citymove.log і в консоль.
- Спільний logger «citymove»: інші модулі пишуть logger.info(...) замість print().
- **@timed** — заміряє час виконання функції (time.perf_counter) і пише в лог; якщо функція впала — логує, скільки вона працювала до помилки, і прокидає виняток далі.
- **@log_call** — логує аргументи виклику і результат; при винятку — повідомлення з traceback.
- **@wraps** — обгортка зберігає ім'я та docstring оригінальної функції.
- Декоратори підключено до обробників CLI; route і stats замість заглушок уже викликають load_network і TransportRouter.
- Тести для CLI і 4 тести для декораторів: успішне виконання та обробка винятку для кожного.

З журналу передачі

Не зроблено: Конфіг-файл з параметрами; @memoize; фіналізація CLI; наскрізний сценарій і скрипт прогону тестів.

Відомі проблеми: Для перевірки handle_route потрібен файл з даними GTFS.

Наступний крок: Тиждень 3: конфіг-файл з параметрами і @memoize.

Новому власнику: Логування перевіряється запуском stats з неіснуючою папкою; лог — у файлі citymove.log.

Тиждень 3 · Роль А: винятки і валідація

Розповідає: Лагута Олександра

domain.py · журнал передачі: docs/HANDOVER-A.txt

- Повна ієрархія винятків: у кожного свій конструктор з повідомленням українською; **StopNotFoundError** зберігає stop_id, **RouteNotFoundError** — зупинки «звідки» і «куди».
- **add_route(route)**: дублікат id → ValueError; якщо хоч однієї зупинки маршруту немає в мережі → StopNotFoundError.
- **add_trip(trip)**: дублікат id → ValueError; маршрут рейсу не зареєстрований у мережі → ScheduleError.
- **Каскадне видалення**: remove_node разом із зупинкою прибирає всі рейси, у розкладі яких вона є, і маршрути, що через неї проходять — у мережі не лишається посилань на зупинку, якої вже немає.
- Порядок у remove_node: спершу перевірка зв'язності (BFS з тижня 2), і лише якщо мережа не розпадається — каскад.
- 4 тести: дублікат зупинки, невідома зупинка, захист від розриву, покрокове видалення з синхронізацією рейсів і маршрутів.

```
NetworkError
 ├─ StopNotFoundError         (stop_id)
 ├─ RouteNotFoundError        (звідки, куди)
 ├─ DisconnectedNetworkError
 └─ ScheduleError
```

З журналу передачі

Не зроблено: Усе заплановане на тиждень 3 зроблено.

Відомі проблеми: Каскад видаляє за прямим збігом посилань на зупинку; для великих імпортів з парсера може знадобитися додаткова обробка часу й розкладу.

Наступний крок: Мокові об'єкти і зв'язка моделі з парсером та консольним сценарієм.

Новому власнику: Усі методи Network самі кидають стандартизовані винятки; їх обробляють вище за стеком.

Тиждень 3 · Роль B: розширений валідатор

Розповідає: Дрижирук Альона

dataio.py · журнал передачі: docs/HANDOVER-B.txt

- Розширено **validate_feed** і **FeedValidationReport** з тижня 2.
- Цілісність **trip_id**: кожен рейс зі stop_times.txt має бути в trips.txt; при strict=True — ScheduleError.
- Зупинки зі stops.txt: координати мають бути числами, lat у [-90, 90], lon у [-180, 180]; порожня назва — зауваження (warning).
- Розклад: перевірка формату часу в stop_times.txt.
- Звіт рахує перевірені сутності (**stats**: зупинки, маршрути, рейси, записи розкладу) і друкується через **format_summary()**.
- Валідатор вбудовано в **load_network** (strict_validation=False): звіт лежить у network.validation_report, а биті рейси не ламають завантаження всієї мережі.
- Тест на фід без обов'язкових файлів.

```
ЗВІТ ВАЛІДАЦІЇ GTFS-ФІДА: ВАЛІДНИЙ (OK)
Критичних помилок (errors): 0     Зауважень (warnings): 0
Перевірено сутностей:
  * stops_count: 1492        * routes_count: 162
  * trips_count: 24393       * stop_times_count: 104123
```

З журналу передачі

Не зроблено: Усе заплановане на тиждень 3 виконано.

Відомі проблеми: Валідатор чекає папку з файлами; zip без розпакування — у Спринті 2.

Наступний крок: Оптимізувати завантаження тестового набору для демонстрації.

Новому власнику: strict_validation=True — сувора перевірка з негайним StopNotFoundError, RouteNotFoundError або ScheduleError.

Тиждень 3 · Роль C: перевірка на синтетичній мережі

Розповідає: Жидик Артем

core.py, tests/test_core.py · журнал передачі: docs/HANDOVER-C.txt

- **generate_radial_ring_network** — генератор маленької тестової мережі за радіально-кільцевою схемою: центр C, чотири периферійні зупинки R1–R4, радіальні рейси C → Ri і кільце R1 → R2 → R3 → R4 → R1 (5 зупинок, 5 рейсів).
- Часи прості — відправлення о 600-й хвилині (10:00), 5 хв між зупинками, тож правильні відповіді рахуються вручну.
- **verify_synthetic_network** проганяє Дейкстру і метрики на цій мережі і порівнює з відомими відповідями — повертає словник «перевірка → True / False»:

Перевірка	Очікується	Результат
C → R3 о 10:00 (штраф 0 і 5)	прибуття 10:10	OK
центральність C / R1	4 / 3	OK
діаметр мережі	2	OK
середня довжина шляху	1.2	OK

- 3 нові юніт-тести (TestSyntheticBenchmark): відомий маршрут, відомі метрики, функція перевірки. Усі тести проєкту проходять.

З журналу передачі

Відомі проблеми: Жодних.

Наступний крок: Демонстраційний скрипт з результатами верифікації та графіком мережі.

Новому власнику: Генератор створює 5 зупинок і 5 рейсів; діаметр = 2, середня довжина шляху = 1.2, центральність: C = 4, R* = 3.

Тиждень 3 · Роль D: графіки в єдиному стилі

Розповідає: Мельникович Ігор

analytics.py, notebooks/report.ipynb · журнал передачі: docs/HANDOVER-D.txt

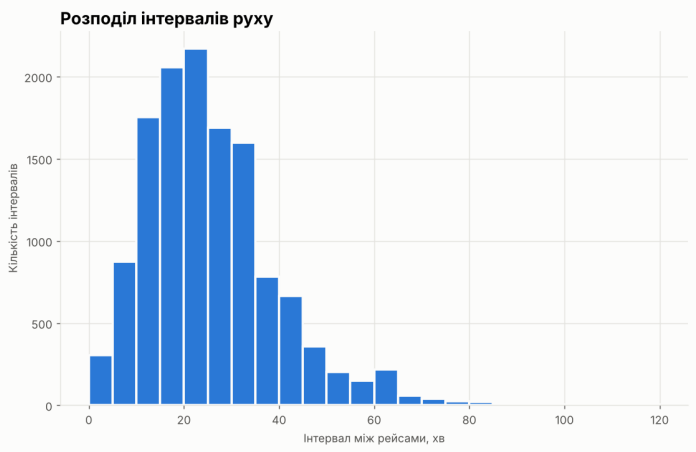
- Закрито пункт «перші два графіки matplotlib в єдиному стилі». Стиль — шаблон для команди: палітра **SERIES_COLORS**, словник **CITYMOVE_STYLE**, **new_axes**(title, xlabel, ylabel) — порожній графік у цьому стилі, **add_legend**(ax) — легенда в тому ж стилі.
- Графік 1 — гістограма інтервалів **plot_headway_histogram**; для неї **route_headways**: інтервали в межах маршруту й напрямку, лише рейси з переданих service_id (будні), нічні розриви > 120 хв відкидаються.
- Графік 2 — Top-10 зупинок за кількістю з'єднань: **busiest_stops** + **plot_busiest_stops**.
- Обидва графіки — у notebooks/report.ipynb на київському фіді. Після мерджу прибрано дублікати графіків з тижня 2 (інший стиль, не використовувались). 6 юніт-тестів.

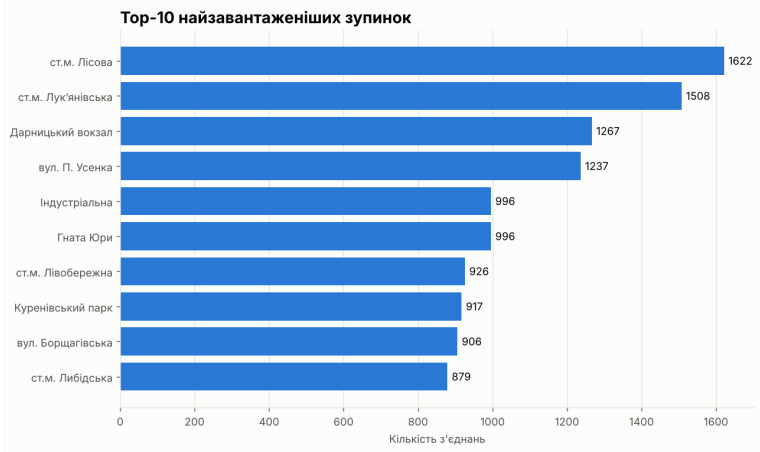
З журналу передачі

Відомі проблеми: Одна зупинка буває записана по-різному («ст.м. Лісова» і «ст.м. "Лісова"») — busiest_stops ігнорує лапки і сумує їх. Network не зберігає calendar.txt, тому будні service_id беруться в ноутбуці через pandas.

Наступний крок: Консольний підсумковий звіт для Демо 1.

Новому власнику: У київському фіді 1240 service_id; без фільтра будні й вихідні змішаються, і інтервали вийдуть заниженими. Нові графіки — лише через new_axes.





Тиждень 3 · Роль E: конфіг, @memoize і прогін тестів

Розповідає: Чайківський Захар

config.json, infra.py, run_tests.py · журнал передачі: docs/HANDOVER-E.txt

- **config.json** — параметри винесено з коду: штраф за пересадку (5 хв), швидкість ходьби (4,5 км/год), максимальна пішохідна відстань (800 м), шлях до фіда, рівень логування.
- **load_config** — читає JSON; немає файлу → попередження в лог і стандартні значення; битий JSON → помилка в лог і порожній словник.
- **@memoize** — кешує результат функції в словнику за ключем «позиційні аргументи + відсортовані kwargs»; повторний виклик з тими самими аргументами одразу повертає збережене — для знайдених шляхів.
- **cache_clear()** — очищує кеш, щоб тести були ізольованими.
- **run_tests.py** — запускає всі тести з папки tests (unittest discover) і повертає код 0 або 1 — придатно для автоматичного запуску.
- Додаткові тести в TestCityMoveCLI: розбір команди draw, невірний формат часу в route, @memoize.

З журналу передачі

Не зроблено: Фіналізація обробки команд CLI.

Відомі проблеми: @memoize зберігає кеш лише в пам'яті під час роботи програми; у тестах скидати кеш через .cache_clear().

Наступний крок: Перевірити роботу всього консольного сценарію CLI перед Демо 1.

```
{
  "transfer_penalty_min": 5,
  "walking_speed_kmh": 4.5,
  "max_walking_distance_m": 800,
  "default_feed_path":
      "data/gtfs_static",
  "log_level": "INFO"
}
```

Тиждень 4 · Роль А: рев'ю і тести

Розповідає: Чайківський Захар

domain.py, tests/test_domain.py · журнал передачі: docs/HANDOVER-A.txt

- Фінальне рев'ю коду ролі А за всі чотири тижні — помилок не виявлено.
- Перевірено покриття тестами всього, що є в domain.py: транспорт (Bus, Tram, Trolleybus, Metro), add_node, remove_node з перевіркою зв'язності (BFS), add_route, add_trip, повна ієрархія винятків.
- Додано 5 тестів:
  - властивості транспорту — лише для читання (змінити не можна);
  - зміна імені пасажира;
  - маршрут без зупинок;
  - add_trip з незареєстрованим маршрутом → ScheduleError;
  - каскадне видалення кількох рейсів разом із зупинкою.
- Код ролі А готовий до Демонстрації 1.

З журналу передачі

Не зроблено: Усі вимоги Спринту 1 для ролі А виконано.

Відомі проблеми: При серіалізації в JSON (Спринт 2) треба не дублювати об'єкти в пам'яті при відновленні.

Наступний крок: Тиждень 5: __iter__ для Route, __len__ і __contains__ для Network, __repr__ і __eq__ для Stop; клас Schedule з next_departure.

Новому власнику: На демо треба вміти пояснити BFS у _is_connected_graph і каскадне видалення рейсів та маршрутів.

Тиждень 4 · Роль B: кеш для швидкого завантаження

Розповідає: Лагута Олександра

dataio.py · журнал передачі: docs/HANDOVER-B.txt

- Бінарний кеш (pickle): **load_network**(folder, use_cache=True) зберігає готовий Network у файл .network_cache.pkl, наступні запуски читають його замість CSV.
- **is_cache_valid** — кеш дійсний, лише якщо він новіший за всі .txt файли фіда (час зміни mtime); змінили будь-який файл — кеш перебудується сам.
- **save_network_cache** / **load_network_cache** — запис і читання кешу.
- **Fallback**: пошкоджений або несумісний кеш → звичайний CSV-парсинг і перезапис валідного кешу.
- RouteNotFoundError тепер приймає або готовий текст, або пару зупинок «звідки — куди».
- Тести: створення кешу, читання з кешу, інвалідація після зміни файлів, пошкоджений кеш.

Завантаження київського фіда	Час
CSV-парсинг (без кешу)	≈ 1,0 с
Перший запуск із кешем (парсинг + запис .pkl)	≈ 1,4 с
Повторний запуск із кешем	≈ 0,16 с (у ~6 разів швидше)

З журналу передачі

Не зроблено: Усе для ролі B за тижні 1–4 виконано.

Відомі проблеми: Якщо змінити класи domain.py, старий pickle може бути несумісним — тоді fallback перепарсить CSV.

Наступний крок: Тиждень 5: генератор iter_stop_times і контекстний менеджер GtfsFeed для zip.

Новому власнику: use_cache за замовчуванням False, щоб тести не створювали файлів; для демо — True. Скинути кеш — видалити .network_cache.pkl.

Тиждень 4 · Роль C: бенчмарк штрафу за пересадку

Розповідає: Дрижирук Альона

core.py, tests/test_core.py · журнал передачі: docs/HANDOVER-C.txt

- **generate_transfer_benchmark_network** — тестова мережа з альтернативними шляхами з A до D:
  - прямий повільний автобус A → B → C → D: 10:00 → 10:30;
  - два експреси A → B (10:00 → 10:08) і B → D (10:10 → 10:18) з пересадкою в B;
  - ізольована зупинка ISO — для перевірки недосяжності.
- Перевіряє критерій вибору Дейкстри «прямий маршрут проти швидкого з пересадкою» залежно від штрафу:

Штраф	Обрано	Прибуття	Вартість	Пересадок
0 хв	два експреси	10:18	618	1
15 хв	прямий	10:30	630 (618 + 15 = 633 > 630)	0

- **format_benchmark_report** — консольний звіт для захисту: верифікація радіально-кільцевої мережі + тест чутливості до штрафу (усе [OK]).
- 4 юніт-тести: зміна вибору від штрафу, пропущена пересадка (рейс уже поїхав), недосяжна ізольована зупинка, вміст звіту.

З журналу передачі

Не зроблено: Усі завдання Спринту 1 для ролі C виконано.

Відомі проблеми: Рейси не фільтруються за днями тижня (service_id).

Наступний крок: Завдання Спринту 2.

Новому власнику: format_benchmark_report() — для швидкої демонстрації на захисті.

Тиждень 4 · Роль D: фіналізація аналітики

Розповідає: Жидик Артем

analytics.py, tests/test_analytics.py · журнал передачі: docs/HANDOVER-D.txt

- Фіналізація і рев'ю analytics.py перед Демо 1.
- **stop_info_table**: виправлено вкладені лапки в f-рядках — у Python 3.11 це синтаксична помилка.
- **departure_board_table**: виправлено виклик StopNotFoundError — передається назва зупинки, щоб виняток коректно зберігав stop_id.
- **console_report_table** — підсумковий звіт рядком (зручно тестувати); generate_console_report друкує і повертає його.
- **route_summary_table** / **print_route_summary** — картка маршруту: тип транспорту, кількість зупинок і рейсів, перший і останній рейс, середній інтервал.
- Аліас **print_depatrures_board** — стара назва з тижня 1 далі працює.
- Тести: з 12 до 19 — звіт для повної й порожньої мережі, замало рейсів для інтервалів, print-функції через перехоплення stdout, route_headways без фільтра service_id.

З журналу передачі

Не зроблено: Усі пункти Спринту 1 для ролі D виконано.

Відомі проблеми: Відсутні; на той момент проходили всі 82 тести проєкту.

Наступний крок: Тиждень 5: Pandas-конвеєр — groupby, merge, pivot_table «маршрут × година», «день тижня × година».

Новому власнику: Звіти — парами: xxx_table() генерує рядок, print_xxx() друкує. Для демо — generate_console_report(network).

Тиждень 4 · Роль E: CLI на реальних даних (1/2)

Розповідає: Мельникович Ігор

cli.py · журнал передачі: docs/HANDOVER-E.txt

- Фіналізовано команди **route, stats, draw** — працюють на реальних даних, без заглушок.
- Параметри: **--feed** для всіх команд (за замовчуванням data/gtfs_static), **--penalty** для route (штраф 5 хв), **--top** для stats.
- **route** приймає stop_id або назву зупинки, перевіряє формат часу HH:MM і друкує ділянки поїздки (вид транспорту, номер, час), прибуття і кількість пересадок.
- **stats** спершу перевіряє фід через validate_feed, потім друкує кількість зупинок, маршрутів і рейсів, діаметр, середню довжину шляху і топ зупинок за центральністю.
- **draw** малює зупинки й лінії маршрутів у PNG (matplotlib).
- **main()** повертає код: 0 — успіх, 1 — помилка в даних (невідома зупинка, немає маршруту, битий фід), 2 — неправильні аргументи.
- --feed і --penalty за замовчуванням беруться з config.json; default_feed_path виправлено на data/gtfs_static, бо data/gtfs.zip не існує.

З журналу передачі

Відомі проблеми: Багато зупинок з однаковою назвою (24 «ст.м. "Лісова"»): береться перша, у лог — WARNING; точніше передавати stop_id. @log_call друкує traceback і для очікуваних помилок. @memoize ще не застосовано: одна команда — один пошук; знадобиться в GUI.

Наступний крок: Спринт 2 — GUI на Tkinter.

Новому власнику: CLI запускається з кореня: python -m src.citymove.cli route --from "ст. Старовокзальна" --to "Мих Борщагівка" --time 06:00

Тиждень 4 · Роль E: наскрізний сценарій (2/2)

Розповідає: Мельникович Ігор

scripts/e2e_demo.py, run_tests.py, tests/test_cli.py · журнал передачі: docs/HANDOVER-E.txt

- **scripts/e2e_demo.py** — наскрізний сценарій: stats → route → draw → навмисна помилка. Запускає CLI окремими процесами і перевіряє коди завершення — тобто перевіряє й обробку помилок.
- Мініфід **tests/fixtures/mini_gtfs** — 4 зупинки і маршрут з однією пересадкою з наперед відомою відповіддю: Вокзал → Парк о 07:55 — автобус до Площі, пересадка на трамвай, прибуття 08:40.
- **run_tests.py** об'єднано з версією тижня 3: працює з будь-якої папки, з прапорцем --e2e додатково запускає наскрізний сценарій.
- Після мерджу з main збережено тести з main (draw, невірний час, @memoize — у ньому бракувало імпорту, через що тест падав); у test_cli.py тепер 22 тести.

```
$ python scripts/e2e_demo.py
>>> 1. Статистика фіда   [OK] код 0
>>> 2. Пошук маршруту
  1. Автобус №1: Вокзал (08:00)
     -> Площа (08:20)
  2. Трамвай №2: Площа (08:30)
     -> Парк (08:40)
  Прибуття: 08:40  Пересадок: 1
                          [OK] код 0
>>> 3. Карта мережі      [OK] код 0
>>> 4. Невідома зупинка
Помилка: Зупинку з ID Неіснуюча
         не знайдено!    [OK] код 1
=== 4/4 кроків успішно ===
```

```
# реальний фід Києва
$ python -m src.citymove.cli route --from "ст. Старовокзальна" \
      --to "Мих Борщагівка" --time 06:00
  1. Трамвай №1: ст. Старовокзальна (06:10) -> Мих Борщагівка (06:46)
Час у дорозі: 46 хв   Пересадок: 0
```

Тести і «зелений стан»

Розповідає: Чайківський Захар

Модуль	Файл тестів	Тестів
domain.py	tests/test_domain.py	15
dataio.py	tests/test_dataio.py	14
core.py	tests/test_core.py	29
analytics.py	tests/test_analytics.py	19
cli.py + infra.py	tests/test_cli.py	22
Разом		**99 — OK**

- python -m unittest discover -s tests — проходить на чистому клоні.
- python run_tests.py — з будь-якої папки; --e2e додає наскрізний сценарій.
- Межові дані: від'ємний штраф, недосяжна зупинка, порожня мережа, битий кеш, некоректний час, видалення зупинки з розривом графа.
- Правильність перевіряється на даних з наперед відомою відповіддю — мініфід і синтетичні мережі.

Як ми працювали

Розповідає: Дрижирук Альона

- Спільний репозиторій; ролі ротувались щотижня (таблиця на слайді 2).
- Гілки feature/<тиждень>-<роль>-<назва> (напр. feature/2-A-domain, feature/3-E-infra); злиття в main через pull request з рев'ю — **21 PR**, 60 комітів (35 без мерджів) за 4 тижні.
- Щотижня — запис у HANDOVER-файлі ролі: що зроблено, що ні, відомі проблеми, наступний крок, як запустити, що знати новому власнику. Новий власник ролі починає саме з нього.
- Конфлікти після мерджу розв'язували явно: у тижні 3 прибрали дублікати графіків з тижня 2, у тижні 4 — відновили пропущений імпорт memoize у тестах CLI.
- Структура за вимогами: src/citymove/{domain, dataio, core, analytics, ui, cli, infra}.py, tests/, data/, notebooks/, docs/ (spec.md + 5 HANDOVER), README.md, .gitignore, requirements.txt.

Відомі обмеження і план на Спринт 2

Розповідає: Лагута Олександра

Що знаємо про поточну версію

- У фіді Київпастрансу кожен stop_id обслуговує лише один маршрут — без пішохідних пересадок граф розпадається на 320 частин (≤ 9 зупинок). Тому пересадки показуємо на мініфіді та синтетиці.
- Пошук ще не фільтрує рейси за днем тижня (service_id).
- Однакові назви зупинок — за назвою береться перша, точніше передавати stop_id.
- @log_call пише traceback і для очікуваних помилок.

Спринт 2 (тижні 5–8)

- Пішохідні пересадки між близькими зупинками (у config.json уже є walking_speed_kmh і max_walking_distance_m).
- A: магічні методи, Schedule.next_departure, будні/вихідні, JSON-серіалізація.
- B: потокове читання, GtfsFeed для zip, генератор мережі, експорт.
- C: матриці NumPy, векторизований Флойд–Воршелл vs Дейкстра, ізохрони 15/30/45 хв.
- D: Pandas-конвеєр, діаграма руху, heatmap доїзду, Jupyter-звіт.
- E: GUI на Tkinter — карта, клік-маршрут, анімація.

Дякуємо!

- Репозиторій: github.com/Artzh13/Citymove
- Запуск: python -m venv venv → pip install -r requirements.txt → python run_tests.py --e2e
- Пошук маршруту: python -m src.citymove.cli route --from Вокзал --to Парк --time 07:55 --feed tests/fixtures/mini_gtfs